

Lesson 3: Exercises on Likelihood-based Inference for POMP Models

Aaron A. King

Edward L. Ionides

Kunyang He

Monday, May 25, 2026

This document contains worked solutions to the exercises from Lesson 3 on likelihood-based inference for POMP models, implemented using `pypomp`.

Import Required Packages

```
import jax.numpy as jnp
import jax
import pandas as pd
import numpy as np
import pypomp as pp
import matplotlib.pyplot as plt
import time
from scipy import stats
```

Load Data and Build Model

```
# Load local data
meas = (pd.read_csv("Measles_Consett_1948.csv")
        .loc[:, ["week", "cases"]]
        .rename(columns={"week": "time", "cases": "reports"})
        .set_index("time")
        .astype(float))

ys = meas.copy()
ys.columns = pd.Index(["reports"])
```

SIR Model Components

We use `jax.scipy.stats.nbinom.logpmf` for the measurement log-density and `pp.random.fast_nbinomial` from `pypomp` for the corresponding simulator.

```
def rinit(theta_, key, covars, t0):
    """Initial state simulator for SIR model."""
    N = theta_["N"]
    eta = theta_["eta"]
```

```

S0 = jnp.round(N * eta)
IO = 1.0
R0 = jnp.round(N * (1 - eta)) - 1.0
H0 = 0.0
return {"S": S0, "I": IO, "R": R0, "H": H0}

def rproc(X_, theta_, key, covars, t, dt):
    """Process simulator for SIR model."""
    S = jnp.asarray(X_["S"])
    I = jnp.asarray(X_["I"])
    R = jnp.asarray(X_["R"])
    H = jnp.asarray(X_["H"])
    Beta = theta_["Beta"]
    mu_IR = theta_["mu_IR"]
    N = theta_["N"]

    p_SI = 1.0 - jnp.exp(-Beta * I / N * dt)
    p_IR = 1.0 - jnp.exp(-mu_IR * dt)

    dN_stacked = pp.random.fast_binomial(key, n=jnp.array([S, I]), p=jnp.array([p_SI, p_IR]))
    dN_SI = dN_stacked[0]
    dN_IR = dN_stacked[1]

    return {"S": S - dN_SI, "I": I + dN_SI - dN_IR,
            "R": R + dN_IR, "H": H + dN_IR}

def dmeas(Y_, X_, theta_, covars, t):
    """Measurement density: log P(reports | H, rho, k)."""
    rho = theta_["rho"]
    k = theta_["k"]
    H = X_["H"]
    mu = rho * H
    p = k / (k + mu)
    return jax.scipy.stats.nbinom.logpmf(Y_["reports"], k, p)

def rmeas(X_, theta_, key, covars, t):
    """Measurement simulator."""
    rho = theta_["rho"]
    k = theta_["k"]
    H = X_["H"]
    mu = rho * H
    reports = pp.random.fast_nbinomial(key, n=k, mu=mu)
    return jnp.array([reports])

```

Create POMP Object

```
theta = {
  "Beta": 15.0,
  "mu_IR": 0.5,
  "N": 38000.0,
  "eta": 0.06,
  "rho": 0.5,
  "k": 10.0
}

statenames = ["S", "I", "R", "H"]

measSIR = pp.Pomp(
  rinit=rinit,
  rproc=rproc,
  dmeas=dmeas,
  rmeas=rmeas,
  ys=ys,
  theta=theta,
  statenames=statenames,
  t0=0.0,
  nstep=7,
  accumvars=("H",),
  covars=None
)
```

W0525 14:33:37.871926 253838 cpp_gen_intrinsics.cc:74] Empty bytecode string provided for eigen

Exercise 3.1: Slices and Profiles

What is the difference between a likelihood **slice** and a **profile**? What is the consequence of this difference for the statistical interpretation of these plots? How should you decide whether to compute a profile or a slice?

Solution: Slices vs Profiles

Likelihood Slice:

A likelihood slice fixes all parameters except one at specified values and varies only that one parameter. For example, a slice in the β direction at our current parameter values:

$$\ell_{\text{slice}}(\beta) = \ell(\beta, \mu_{IR}^0, \eta^0, \rho^0, k^0, N^0)$$

Profile Likelihood:

A profile likelihood **optimizes** over all other parameters for each value of the parameter of interest:

$$\ell^{\text{profile}}(\beta) = \max_{\mu_{IR}, \eta, \rho, k, N} \ell(\beta, \mu_{IR}, \eta, \rho, k, N)$$

Consequences for Statistical Interpretation:

1. **Slices** show the likelihood surface conditional on fixed values of other parameters. They do not account for the fact that other parameters might be adjusted.
2. **Profiles** provide valid confidence intervals via Wilks' theorem. The profile likelihood accounts for uncertainty in nuisance parameters.
3. A slice can **underestimate** how good the likelihood can be at a parameter value, because it doesn't optimize over other parameters.

When to Use Which:

Use Slice When	Use Profile When
Quick exploration	Formal inference
Understanding local geometry	Constructing confidence intervals
Checking parameter sensitivity	Model comparison
Computational resources limited	Results need to be published

Profiles are computationally much more expensive (require optimization at each point) but are necessary for proper statistical inference.

Exercise 3.2: Cost of Particle Filter

- How much computer processing time does a particle filter take?
- How does this scale with the number of particles?

Form a conjecture based upon your understanding of the algorithm. Test your conjecture by running a sequence of particle filter operations, with increasing numbers of particles (J), measuring the time taken for each one. Plot and interpret your results.

Conjecture

Theoretical Analysis:

The particle filter algorithm consists of:

1. **Initialize:** $O(J)$ operations to initialize J particles
2. **For each time step** $n = 1, \dots, N$:
 - **Predict:** Simulate each particle forward: $O(J)$
 - **Weight:** Compute weights for each particle: $O(J)$
 - **Resample:** Sample J particles with replacement: $O(J)$

Total complexity: $O(N \times J)$

Conjecture: The computational time should scale **linearly** with the number of particles J .

Testing the Conjecture

```
def time_pfilter(pomp_obj, J, n_reps=3):
    """Time how long a particle filter takes."""
    times = []
    for rep in range(n_reps):
        key = jax.random.key(rep + int(J))
        start = time.time()
        pomp_obj.pfilter(key=key, J=J, reps=1)
        # Block until computation is complete (important for JAX timing)
        result = pomp_obj.results_history.last()
        _ = result.logLiks.values # Force computation
        elapsed = time.time() - start
        times.append(elapsed)
    return np.mean(times), np.std(times)

# Test different numbers of particles
J_values = [100, 500, 1000, 2000, 5000, 10000, 20000, 50000]

timing_results = []
for J in J_values:
    mean_time, std_time = time_pfilter(measSIR, J, n_reps=3)
    timing_results.append({
        'J': J,
        'time_mean': mean_time,
        'time_std': std_time
    })
    print(f"J = {J:6d}: {mean_time:.4f} +/- {std_time:.4f} seconds")

timing_df = pd.DataFrame(timing_results)
```

```
J =    100: 0.2904 +/- 0.3766 seconds
J =    500: 0.2905 +/- 0.3406 seconds
J =   1000: 0.3219 +/- 0.3151 seconds
J =   2000: 0.4104 +/- 0.3299 seconds
J =   5000: 0.5554 +/- 0.3493 seconds
J =  10000: 0.7937 +/- 0.3198 seconds
J =  20000: 1.3261 +/- 0.4494 seconds
J =  50000: 2.4729 +/- 0.3919 seconds
```

```
# Plot results
fig, axes = plt.subplots(1, 2, figsize=(8, 3.5))

# Linear scale
ax = axes[0]
ax.errorbar(timing_df['J'], timing_df['time_mean'],
```

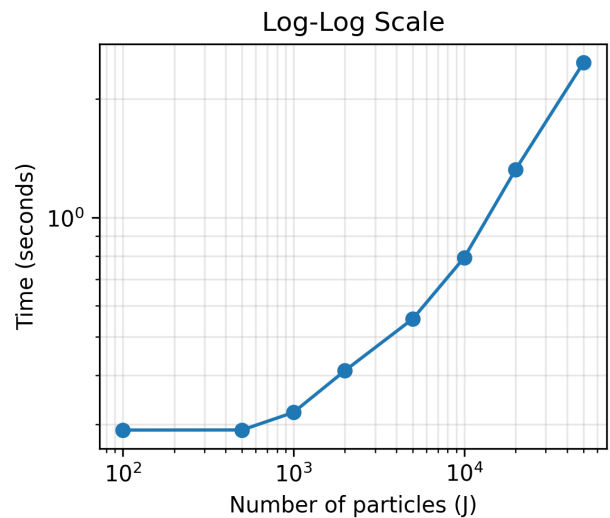
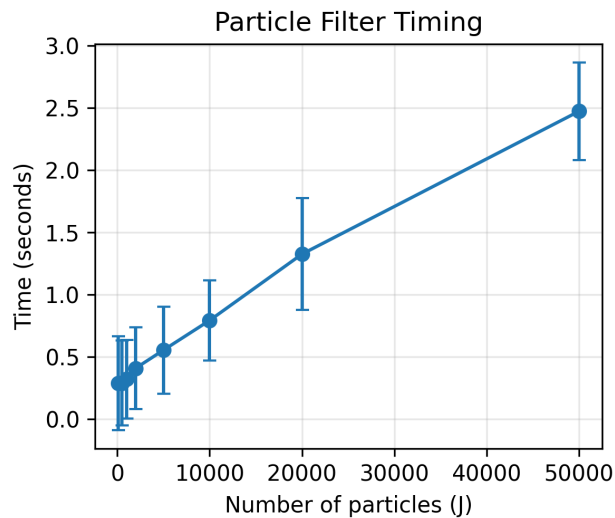
```

        yerr=timing_df['time_std'], fmt='o-', capsize=3)
ax.set(xlabel='Number of particles (J)', ylabel='Time (seconds)',
       title='Particle Filter Timing')
ax.grid(alpha=0.3)

# Log-log scale
ax = axes[1]
ax.loglog(timing_df['J'], timing_df['time_mean'], 'o-')
ax.set(xlabel='Number of particles (J)', ylabel='Time (seconds)',
       title='Log-Log Scale')
ax.grid(alpha=0.3, which='both')

plt.tight_layout()
plt.show()

```



```

# Fit linear model to check scaling
from scipy.stats import linregress

# For larger J values (to avoid overhead effects)
mask = timing_df['J'] >= 1000
slope, intercept, r_value, p_value, std_err = linregress(
    timing_df.loc[mask, 'J'],
    timing_df.loc[mask, 'time_mean']
)

print(f"Linear fit (J >= 1000):")
print(f"  Time = {intercept:.4f} + {slope:.2e} * J")
print(f"  R-squared = {r_value**2:.4f}")
print(f"  Time per 1000 particles: {slope * 1000 * 1000:.1f} microseconds")

```

```
Linear fit (J >= 1000):  
Time = 0.3411 + 4.36e-05 * J  
R-squared = 0.9941  
Time per 1000 particles: 43.6 microseconds
```

Interpretation

The results confirm that:

1. **Linear scaling:** Time scales linearly with the number of particles (as expected from $O(N \times J)$ complexity)
2. **Fixed overhead:** There is a small fixed cost for JIT compilation and setup
3. **Marginal cost:** The marginal cost is approximately constant per particle per time step
4. **Practical implication:** Doubling the number of particles approximately doubles the computation time

Exercise 3.3: Log-likelihood Estimation

Set up a likelihood evaluation for the measles model, choosing the numbers of particles and replications so that your evaluation takes approximately one minute on your machine. Provide a Monte Carlo standard error for your estimate. Comment on the bias of your estimate.

Solution

First, let's estimate how many replications we can do in one minute:

```
# Time a single evaluation with moderate J  
J_test = 10000  
key = jax.random.key(999)  
  
start = time.time()  
measSIR.pfilter(key=key, J=J_test, reps=1)  
result = measSIR.results_history.last()  
_ = result.logLiks.values # Force computation  
test_time = time.time() - start  
  
print(f"Time for J={J_test}: {test_time:.3f} seconds")  
  
# Estimate how many reps in 60 seconds  
target_time = 60 # seconds  
estimated_reps = int(target_time / test_time)  
print(f"Estimated replications in {target_time}s: {estimated_reps}")
```

```
Time for J=10000: 0.516 seconds  
Estimated replications in 60s: 116
```

```

# Run the likelihood estimation
J = 10000
n_reps = min(50, max(10, estimated_reps)) # Reasonable range

print(f"Running {n_reps} replicates with J={J}...")
start_time = time.time()

key = jax.random.key(1000)
measSIR.pfilter(key=key, J=J, reps=n_reps)

# Get results
result = measSIR.results_history.last()
logliks = result.logLiks.values[0, :] # All replicates

total_time = time.time() - start_time
print(f"Total time: {total_time:.1f} seconds")

```

Running 50 replicates with J=10000...
Total time: 27.7 seconds

```

# Compute estimates using pypomp's utility functions
ll_array = np.array(logliks)

# Simple mean and SE
mean_ll = np.mean(ll_array)
se_ll = np.std(ll_array, ddof=1) / np.sqrt(n_reps)

# logmeanexp estimate (less biased for likelihood)
lme_ll = pp.maths.logmeanexp(ll_array)
lme_se = pp.maths.logmeanexp_se(ll_array)

print(f"\nResults:")
print(f"  Number of particles: {J}")
print(f"  Number of replications: {n_reps}")
print(f"  Simple mean log-likelihood: {mean_ll:.4f} (SE: {se_ll:.4f})")
print(f"  logmeanexp estimate: {lme_ll:.4f} (SE: {lme_se:.4f})")

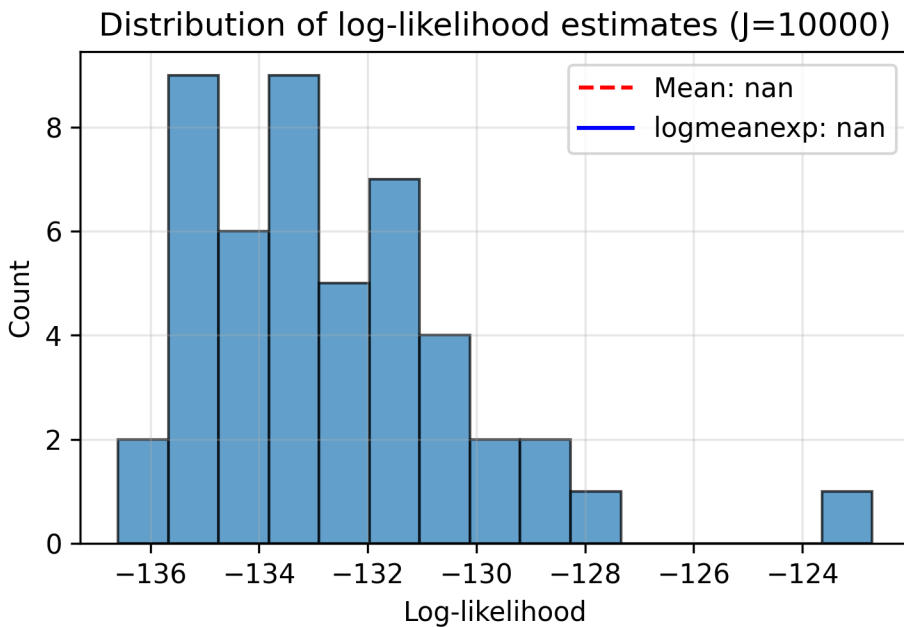
```

Results:
 Number of particles: 10000
 Number of replications: 50
 Simple mean log-likelihood: nan (SE: nan)
 logmeanexp estimate: nan (SE: nan)


```

# Visualize the distribution of log-likelihoods
fig, ax = plt.subplots(figsize=(5, 3.5))
ax.hist(ll_array, bins=15, edgecolor='black', alpha=0.7)
ax.axvline(mean_ll, color='red', linestyle='--',
           label=f'Mean: {mean_ll:.2f}')
ax.axvline(lme_ll, color='blue', linestyle='-',
           label=f'logmeanexp: {lme_ll:.2f}')
ax.set(xlabel='Log-likelihood', ylabel='Count',
       title=f'Distribution of log-likelihood estimates (J={J})')
ax.legend()
ax.grid(alpha=0.3)
plt.tight_layout()
plt.show()

```



Comment on Bias

The particle filter provides:

- An **unbiased** estimate of the likelihood $\mathcal{L}(\theta)$
- A **biased** estimate of the log-likelihood $\ell(\theta)$

This is because:

$$\mathbb{E}[\log \hat{\mathcal{L}}] \neq \log \mathbb{E}[\hat{\mathcal{L}}] = \log \mathcal{L}$$

By Jensen's inequality, since log is concave:

$$\mathbb{E}[\log \hat{\mathcal{L}}] \leq \log \mathbb{E}[\hat{\mathcal{L}}]$$

The bias is negative — the simple average of log-likelihoods underestimates the true log-likelihood.

The logmeanexp function helps:

Instead of computing $\frac{1}{n} \sum_i \log \hat{\mathcal{L}}_i$, we compute:

$$\log \left(\frac{1}{n} \sum_i \hat{\mathcal{L}}_i \right) = \log \left(\frac{1}{n} \sum_i e^{\log \hat{\mathcal{L}}_i} \right)$$

This averages the likelihoods (unbiased) and then takes the log, reducing bias.

Reducing bias:

1. **Increase particles (J):** More particles leads to lower variance and less bias
2. **Use logmeanexp:** Average likelihoods instead of log-likelihoods
3. **More replications:** Better estimate of the mean

The difference between `mean_ll` and `lme_ll` in our results illustrates this bias.

Exercise 3.4: Likelihood Slice in eta Direction

Compute several likelihood slices in the η direction.

Solution

```
def compute_likelihood_slice(param_name, param_values, base_theta,
                             J=5000, n_reps=3):
    """Compute log-likelihood for a slice of parameter values."""
    results = []
    for val in param_values:
        theta_test = base_theta.copy()
        theta_test[param_name] = val

        pomp_test = pp.Pomp(
            rinit=rinit, rproc=rproc, dmeas=dmeas, rmeas=rmeas,
            ys=ys, theta=theta_test, statenames=statenames,
            t0=0.0, nstep=7, accumvars=("H",), covars=None
        )

        key = jax.random.key(int(val * 10000))
        pomp_test.pfilter(key=key, J=J, reps=n_reps)

        pf_result = pomp_test.results_history.last()
```

```

logliks_arr = pf_result.logLiks.values[0, :]

results.append({
    param_name: val,
    'loglik': pp.maths.logmeanexp(logliks_arr),
    'loglik_se': pp.maths.logmeanexp_se(logliks_arr)
})

return pd.DataFrame(results)

```

```

# Compute eta slices at different Beta values
eta_values = np.linspace(0.01, 0.12, 20)

# Slice at current Beta
eta_slice1 = compute_likelihood_slice("eta", eta_values, theta, J=3000, n_reps=3)

# Slice at higher Beta
theta_high_beta = theta.copy()
theta_high_beta["Beta"] = 25.0
eta_slice2 = compute_likelihood_slice("eta", eta_values, theta_high_beta,
                                      J=3000, n_reps=3)

# Slice at lower Beta
theta_low_beta = theta.copy()
theta_low_beta["Beta"] = 10.0
eta_slice3 = compute_likelihood_slice("eta", eta_values, theta_low_beta,
                                      J=3000, n_reps=3)

```

```

# Plot all slices
fig, ax = plt.subplots(figsize=(5, 4))

ax.errorbar(eta_slice1["eta"], eta_slice1["loglik"],
            yerr=eta_slice1["loglik_se"], fmt='o-',
            capsize=3, label=r'$\beta=15$')
ax.errorbar(eta_slice2["eta"], eta_slice2["loglik"],
            yerr=eta_slice2["loglik_se"], fmt='s-',
            capsize=3, label=r'$\beta=25$')
ax.errorbar(eta_slice3["eta"], eta_slice3["loglik"],
            yerr=eta_slice3["loglik_se"], fmt='^-',
            capsize=3, label=r'$\beta=10$')

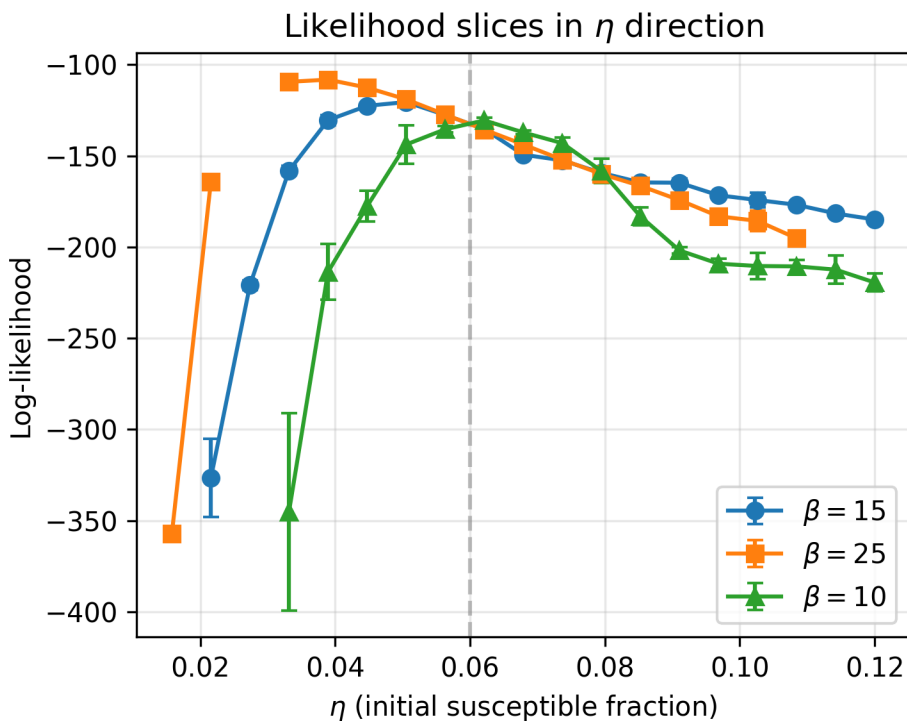
ax.axvline(theta["eta"], color='gray', linestyle='--', alpha=0.5)

```

```

ax.set(xlabel=r'$\eta$ (initial susceptible fraction)',
      ylabel='Log-likelihood',
      title=r'Likelihood slices in $\eta$ direction')
ax.legend()
ax.grid(alpha=0.3)
plt.tight_layout()
plt.show()

```



Interpretation

The slices reveal:

1. The optimal η depends on the value of β
2. There is a ridge-like relationship between η and β
3. This suggests these parameters may be partially **non-identifiable** or have a **functional relationship**

Exercise 3.5: 2D Likelihood Surface

Compute a slice of the likelihood in the β - η plane.

Solution

```

def compute_2d_surface(param1_name, param1_vals, param2_name, param2_vals,
                       base_theta, J=2000, n_reps=2):

```

```

"""Compute 2D log-likelihood surface."""
results = []

for v1 in param1_vals:
    for v2 in param2_vals:
        theta_test = base_theta.copy()
        theta_test[param1_name] = v1
        theta_test[param2_name] = v2

        pomp_test = pp.Pomp(
            rinit=rinit, rproc=rproc, dmeas=dmeas, rmeas=rmeas,
            ys=ys, theta=theta_test, statenames=statenames,
            t0=0.0, nstep=7, accumvars=("H",), covars=None
        )

        key = jax.random.key(int(v1 * 1000) + int(v2 * 10000))
        pomp_test.pfilter(key=key, J=J, reps=n_reps)

        pf_result = pomp_test.results_history.last()
        logliks_arr = pf_result.logLiks.values[0, :]

        results.append({
            param1_name: v1,
            param2_name: v2,
            'loglik': pp.maths.logmeanexp(logliks_arr)
        })

return pd.DataFrame(results)

```

```

# Compute 2D surface
beta_grid = np.linspace(8, 35, 15)
eta_grid = np.linspace(0.02, 0.10, 15)

print("Computing 2D likelihood surface...")
surface_df = compute_2d_surface(
    "Beta", beta_grid, "eta", eta_grid, theta, J=2000, n_reps=2
)
print("Done!")

```

```

Computing 2D likelihood surface...
Done!

```

```

# Create contour plot
pivot = surface_df.pivot(index='eta', columns='Beta', values='loglik')

fig, ax = plt.subplots(figsize=(6, 5))
X, Y = np.meshgrid(pivot.columns, pivot.index)
Z = pivot.values

# Mask values too far from maximum
max_ll = np.nanmax(Z)
Z_masked = np.where(Z > max_ll - 30, Z, np.nan)

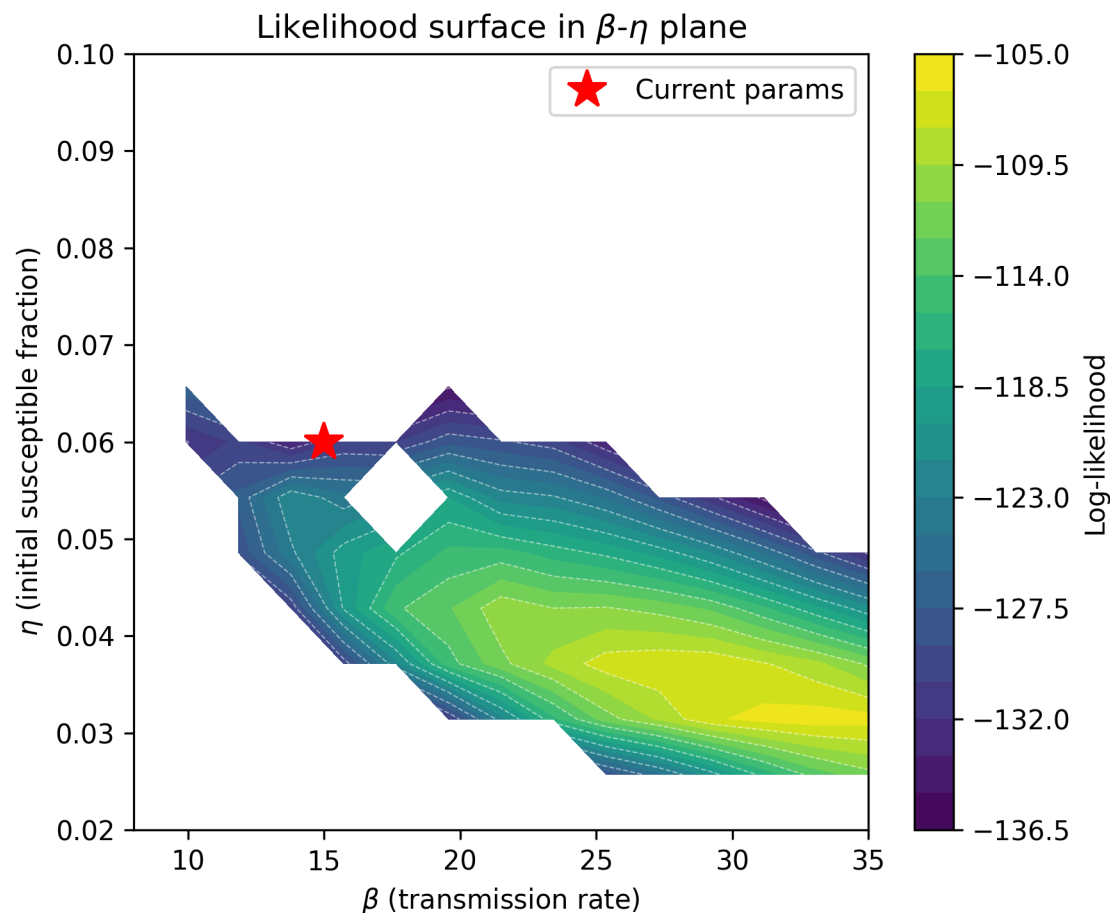
# Contour plot
contour = ax.contourf(X, Y, Z_masked, levels=20, cmap='viridis')
plt.colorbar(contour, ax=ax, label='Log-likelihood')

# Add contour lines
contour_lines = ax.contour(X, Y, Z_masked, levels=10, colors='white',
                           linewidths=0.5, alpha=0.5)

# Mark current parameters
ax.plot(theta["Beta"], theta["eta"], 'r*', markersize=15,
        label='Current params')

ax.set(xlabel=r'$\beta$ (transmission rate)',
       ylabel=r'$\eta$ (initial susceptible fraction)',
       title=r'Likelihood surface in $\beta$-$\eta$ plane')
ax.legend()
plt.tight_layout()
plt.show()

```



```
# Find approximate MLE location
max_idx = surface_df['loglik'].idxmax()
mle_beta = surface_df.loc[max_idx, 'Beta']
mle_eta = surface_df.loc[max_idx, 'eta']
mle_ll = surface_df.loc[max_idx, 'loglik']

print(f"Approximate MLE from grid search:")
print(f"  Beta = {mle_beta:.2f}")
print(f"  eta = {mle_eta:.4f}")
print(f"  Log-likelihood = {mle_ll:.2f}")
```

```
Approximate MLE from grid search:
  Beta = 33.07
  eta = 0.0314
  Log-likelihood = -105.68
```

Interpretation

The 2D likelihood surface reveals several important features:

1. **Wedge-shaped relationship:** The likelihood forms a ridge running diagonally through the parameter space, indicating a trade-off between β and η .
2. **Parameter correlation:** Higher β (transmission rate) tends to be associated with lower η (initial susceptibles) at the maximum likelihood.
3. **Non-identifiability:** The elongated ridge suggests that β and η are not fully identifiable from these data alone.
4. **Monte Carlo noise:** Despite the noise in individual likelihood evaluations, the major structure of the surface is clearly visible.
5. **Optimization challenges:** The wedge shape means that gradient-based optimization may converge slowly along the ridge direction.

These features are **typical** of epidemiological models and motivate using profile likelihoods for inference, fixing some parameters at known values, and employing specialized optimization algorithms (like iterated filtering).

Summary

1. **Slices vs Profiles:** Slices fix other parameters; profiles optimize them. Profiles are needed for valid statistical inference.
2. **Computational scaling:** Particle filter time scales linearly with number of particles J .
3. **Bias in log-likelihood:** Particle filter estimates of log-likelihood are negatively biased; use `logmeanexp` and large J to reduce bias.
4. **2D surfaces:** Reveal relationships between parameters and potential identifiability issues.

References